shown in the following *mapper.py*:

```
#!/usr/bin/python
import sys
import re
# Generator for words in a line
def read_input(file):
  for line in file:
    yield re.compile('\W*').split(line)

def main(sep='\t'):
  for words in read_input(sys.stdin):
    wd_dict={}
# count the number of occurences of a case-insensitive word
    for word in words:
      if word != '':
        wd = word.lower()
        wd_dict[wd] = wd_dict.get(wd,0) + 1
# print each word and its count
    for word in wd_dict:
        print('%s%s%d'%(word,sep,wd_dict[word]))
if __name__ == "__main__" :
  main()
```

There is no need to change the reducer function as long as it had not assumed that the count of each word was 1. The corresponding *reducer.py:*

```
#!/usr/bin/python
import sys
# convert each line into a word and its count
def read_mapper_output(file, sep):
  for line in file:
    yield line.strip().split(sep)
# add the count for each occurrence of the word
def main(sep='\t'):
  wc = {}
  for word, count in read_mapper_output(sys.stdin, sep):
    wc[word] = wc.get(word,0) + int(count)
# print the sorted list
  for word in sorted(wc):
    print("%s%s%d"%(word,sep,wc[word]))
if __name__ == "__main__":
  main()
```

So, having signed into *fedora@h-mstr*, you may run these commands to count the words and examine the result:

```
$ hadoop jar /usr/share/java/hadoop/hadoop-streaming.jar \
    -files mapper.py,reducer.py   -mapper mapper.py  -reducer
reducer.py\
    -input document_files.txt -output wordcount.out
$ hadoop fs -cat wordcount.out/part-00000 | less
```

As seen in the earlier experiments, you can store the data in a HDFS file using any programming language. Then, you can write fairly simple 'map and reduce' programs in any programming language, without worrying about any issues related to distributed processing. Hadoop will make the effort to optimise the distribution of the data across the nodes as well as feed the data to the appropriate mapper programs. END